

```

        if (jstack >= NSTACK) throw("NSTACK too small in sort.");
        if (ir-i+1 >= j-1) {
            istack[jstack]=ir;
            istack[jstack-1]=i;
            ir=j-1;
        } else {
            istack[jstack]=j-1;
            istack[jstack-1]=l;
            l=i;
        }
    }
}
}

```

As usual, you can move any other arrays around at the same time as you sort `arr`. At the risk of being repetitious:

```

template<class T, class U>
void sort2(NRvector<T> &arr, NRvector<U> &brr)
Sort an array arr[0..n-1] into ascending order using Quicksort, while making the corresponding
rearrangement of the array brr[0..n-1].
{
    const Int M=7,NSTACK=64;
    Int i,ir,j,k,jstack=-1,l=0,n=arr.size();
    T a;
    U b;
    VecInt istack(NSTACK);
    ir=n-1;
    for (;;) {
        if (ir-l < M) {
            for (j=l+1;j<=ir;j++) {
                a=arr[j];
                b=brr[j];
                for (i=j-1;i>=l;i--) {
                    if (arr[i] <= a) break;
                    arr[i+1]=arr[i];
                    brr[i+1]=brr[i];
                }
                arr[i+1]=a;
                brr[i+1]=b;
            }
            if (jstack < 0) break;
            ir=istack[jstack--];
            l=istack[jstack--];
        } else {
            k=(l+ir) >> 1;
            SWAP(arr[k],arr[l+1]);
            SWAP(brr[k],brr[l+1]);
            if (arr[l] > arr[ir]) {
                SWAP(arr[l],arr[ir]);
                SWAP(brr[l],brr[ir]);
            }
            if (arr[l+1] > arr[ir]) {
                SWAP(arr[l+1],arr[ir]);
                SWAP(brr[l+1],brr[ir]);
            }
            if (arr[l] > arr[l+1]) {
                SWAP(arr[l],arr[l+1]);
                SWAP(brr[l],brr[l+1]);
            }
            i=l+1;
            j=ir;
            a=arr[l+1];

```

sort.h

Insertion sort when subarray small enough.

Pop stack and begin a new round of partitioning.

Choose median of left, center, and right elements as partitioning element `a`. Also rearrange so that $a[l] \leq a[l+1] \leq a[ir]$.

Initialize pointers for partitioning.

Partitioning element.

```

b=brr[l+1];
for (;;) {
    do i++; while (arr[i] < a);
    do j--; while (arr[j] > a);
    if (j < i) break;
    SWAP(arr[i],arr[j]);
    SWAP(brr[i],brr[j]);
}
arr[l+1]=arr[j];
arr[j]=a;
brr[l+1]=brr[j];
brr[j]=b;
jstack += 2;
Push pointers to larger subarray on stack; process smaller subarray immediately.
if (jstack >= NSTACK) throw("NSTACK too small in sort2.");
if (ir-i+1 >= j-1) {
    istack[jstack]=ir;
    istack[jstack-1]=i;
    ir=j-1;
} else {
    istack[jstack]=j-1;
    istack[jstack-1]=l;
    l=i;
}
}
}

```

Beginning of innermost loop.
 Scan up to find element > a.
 Scan down to find element < a.
 Pointers crossed. Partitioning complete.
 Exchange elements of both arrays.
 End of innermost loop.
 Insert partitioning element in both arrays.

You could, in principle, rearrange any number of additional arrays along with `brr`, but this is inefficient if the number of such arrays is larger than one. The preferred technique is to make use of an index table, as described in §8.4.

CITED REFERENCES AND FURTHER READING:

Sedgewick, R. 1978, "Implementing Quicksort Programs," *Communications of the ACM*, vol. 21, pp. 847–857.[1]

8.3 Heapsort

Heapsort is slower than Quicksort by a constant factor. It is so beautiful that we sometimes use it anyway, just for the sheer joy of it. (However, we don't recommend that you do this if your employer is paying for efficient code.) Heapsort is a true "in-place" sort, requiring no auxiliary storage. It is an $N \log_2 N$ algorithm, not only on average, but also for the worst case order of input data. In fact, its worst case is only 20 % or so worse than its average running time.

It is beyond our scope to give a complete exposition on the theory of Heapsort. We mention the general principles, then refer you to the references [1,2]; or you can analyze the program yourself, if you want to understand the details.

A set of N numbers a_j , $j = 0, \dots, N - 1$, is said to form a "heap" if it satisfies the relation

$$a_{(j-1)/2} \geq a_j \quad \text{for} \quad 0 \leq (j-1)/2 < j < N \quad (8.3.1)$$

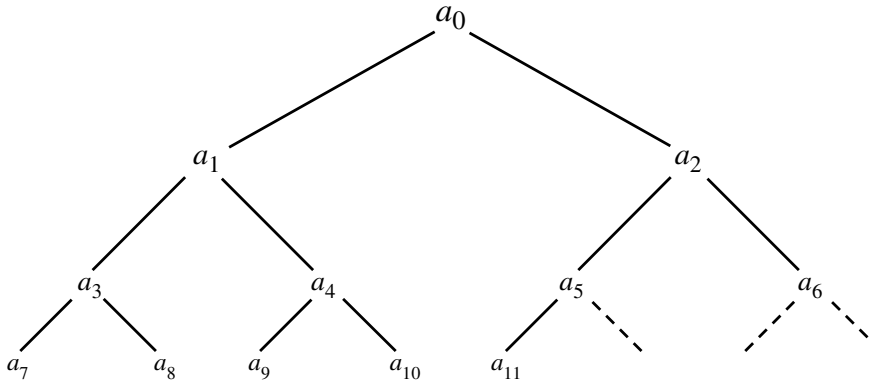


Figure 8.3.1. Ordering implied by a “heap,” here of 12 elements. Elements connected by an upward path are sorted with respect to one another, but there is not necessarily any ordering among elements related only “laterally.”

Here the division in $j/2$ means “integer divide,” i.e., is an exact integer or else is rounded down to the closest integer. Definition (8.3.1) will make sense if you think of the numbers a_i as being arranged in a binary tree, with the top, “boss,” node being a_0 ; the two “underling” nodes being a_1 and a_2 ; *their* four underling nodes being a_3 through a_6 ; etc. (See Figure 8.3.1.) In this form, a heap has every “supervisor” greater than or equal to its two “supervisees,” down through the levels of the hierarchy.

If you have managed to rearrange your array into an order that forms a heap, then sorting it is very easy: You pull off the “top of the heap,” which will be the largest element yet unsorted. Then you “promote” to the top of the heap its largest underling. Then you promote *its* largest underling, and so on. The process is like what happens (or is supposed to happen) in a large corporation when the chairman of the board retires. You then repeat the whole process by retiring the new chairman of the board. Evidently the whole thing is an $N \log_2 N$ process, since each retiring chairman leads to $\log_2 N$ promotions of underlings.

Well, how do you arrange the array into a heap in the first place? The answer is again a “sift-up” process like corporate promotion. Imagine that the corporation starts out with $N/2$ employees on the production line, but with no supervisors. Now a supervisor is hired to supervise two workers. If he is less capable than one of his workers, that one is promoted in his place, and he joins the production line. After supervisors are hired, then supervisors of supervisors are hired, and so on up the corporate ladder. Each employee is brought in at the top of the tree, but then immediately sifted down, with more capable workers promoted until their proper corporate level has been reached.

In the Heapsort implementation, the same sift-up code can be used for the initial creation of the heap and for the subsequent retirement-and-promotion phase. One execution of the Heapsort function represents the entire life-cycle of a giant corporation: $N/2$ workers are hired; $N/2$ potential supervisors are hired; there is a sifting up in the ranks, a sort of super Peter Principle: In due course, each of the original employees gets promoted to chairman of the board.

```

sort.h namespace hpsort_util {
    template<class T>
    void sift_down(NRvector<T> &ra, const Int l, const Int r)
    Carry out the sift-down on element ra(l) to maintain the heap structure. l and r determine
    the "left" and "right" range of the sift-down.
    {
        Int j,jold;
        T a;
        a=ra[l];
        jold=l;
        j=2*l+1;
        while (j <= r) {
            if (j < r && ra[j] < ra[j+1]) j++;
            if (a >= ra[j]) break;
            ra[jold]=ra[j];
            jold=j;
            j=2*j+1;
        }
        ra[jold]=a;
    }
}

template<class T>
void hpsort(NRvector<T> &ra)
Sort an array ra[0..n-1] into ascending numerical order using the Heapsort algorithm. ra is
replaced on output by its sorted rearrangement.
{
    Int i,n=ra.size();
    for (i=n/2-1; i>=0; i--)
        The index i, which here determines the "left" range of the sift-down, i.e., the element
        to be sifted down, is decremented from n/2-1 down to 0 during the "hiring" (heap
        creation) phase.
        hpsort_util::sift_down(ra,i,n-1);
    for (i=n-1; i>0; i--) {
        Here the "right" range of the sift-down is decremented from n-2 down to 0 during the
        "retirement-and-promotion" (heap selection) phase.
        SWAP(ra[0],ra[i]);
        Clear a space at the end of the array, and retire
        hpsort_util::sift_down(ra,0,i-1); the top of the heap into it.
    }
}

```

CITED REFERENCES AND FURTHER READING:

Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §5.2.3.[1]
 Sedgewick, R. 1998, *Algorithms in C*, 3rd ed. (Reading, MA: Addison-Wesley), Chapter 11.[2]

8.4 Indexing and Ranking

The concept of *keys* plays a prominent role in the management of data files. A data *record* in such a file may contain several items, or *fields*. For example, a record in a file of weather observations may have fields recording time, temperature, and wind velocity. When we sort the records, we must decide which of these fields we want to be brought into sorted order. The other fields in a record just come along for the ride and will not, in general, end up in any particular order. The field on which the sort is performed is called the *key* field.

For a data file with many records and many fields, the actual movement of N records into the sorted order of their keys K_i , $i = 0, \dots, N - 1$, can be a daunting task. Instead, one can construct an *index table* I_j , $j = 0, \dots, N - 1$, such that the smallest K_i has $i = I_0$, the second smallest has $i = I_1$, and so on up to the largest K_i with $i = I_{N-1}$. In other words, the array

$$K_{I_j} \quad j = 0, 1, \dots, N - 1 \quad (8.4.1)$$

is in sorted order when indexed by j . When an index table is available, one need not move records from their original order. Further, different index tables can be made from the same set of records, indexing them to different keys.

The algorithm for constructing an index table is straightforward: Initialize the index array with the integers from 0 to $N - 1$; then perform the Quicksort algorithm, moving the elements around *as if* one were sorting the keys. The integer that initially numbered the smallest key thus ends up in the number one position, and so on.

The concept of an index table maps particularly nicely into an object, say `Indexx`. The constructor takes a vector `arr` as its argument; it stores an index table to `arr`, leaving `arr` unmodified. Subsequently, the method `sort` can be invoked to rearrange `arr`, or any other vector, into the sorted order of `arr`. `Indexx` is not a templated class, since the stored index table does not depend on the type of vector that is indexed. However, it does need a templated constructor.

```
struct Indexx { sort.h
    Int n;
    VecInt indx;

    template<class T> Indexx(const NRvector<T> &arr) {
        Constructor. Calls index and stores an index to the array arr[0..n-1].
        index(&arr[0], arr.size());
    }
    Indexx() {} Empty constructor. See text.

    template<class T> void sort(NRvector<T> &brr) {
        Sort an array brr[0..n-1] into the order defined by the stored index. brr is replaced on
        output by its sorted rearrangement.
        if (brr.size() != n) throw("bad size in Index sort");
        NRvector<T> tmp(brr);
        for (Int j=0; j<n; j++) brr[j] = tmp[indx[j]];
    }

    template<class T> inline const T & el(NRvector<T> &brr, Int j) const {
        This function, and the next, return the element of brr that would be in sorted position j
        according to the stored index. The vector brr is not changed.
        return brr[indx[j]];
    }
    template<class T> inline T & el(NRvector<T> &brr, Int j) {
        Same, but return an l-value.
        return brr[indx[j]];
    }

    template<class T> void index(const T *arr, Int nn);
    This does the actual work of indexing. Normally not called directly by the user, but see
    text for exceptions.

    void rank(VecInt_0 &irank) {
        Returns a rank table, whose jth element is the rank of arr[j], where arr is the vector
        originally indexed. The smallest arr[j] has rank 0.
        irank.resize(n);
```